

Accommodation Allocation Engine

Optimal hostel-room assignment for the cultural festival

Track 1 — Mathematical Models for Operations

Hungarian (Kuhn–Munkres) algorithm + bipartite matching

Group <No.> · OOP Summer 2026 · BITS Pilani

Members: <Name 1>, <Name 2>, <Name 3>, <Name 4>

1 · The problem

- The festival hosts **5,000–6,000 users**; outstation participants need **hostel rooms**.
- Today allocation is **manual** → clashes, gender/accessibility errors, ignored preferences, under-used beds, and a chaotic waitlist.
- Rooms are a **scarce, constrained resource**: fixed capacity, gender policy, price, accessibility.

Goal: automatically assign participants to rooms that **minimises total dissatisfaction** while **never violating a hard constraint**, and fairly waitlist the overflow.

2 · Inputs & outputs

Inputs (from the Accommodation module, CSV/JSON):

- *Participants*: id, gender, budget/night, nights, arrival day, accessibility need, **category** (Performer/VIP/Delegate/Attendee), preferences (building, room type, roommates).
- *Rooms*: id, building, floor, **capacity**, **gender policy**, price/night, accessible?, type.

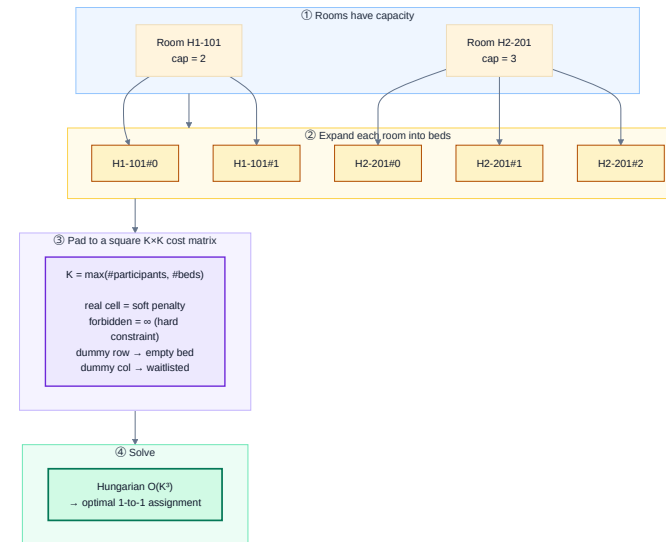
Outputs:

- `allocations` — participant → room + **wallet charge** (price × nights).
- `waitlist` — overflow, ordered by priority.
- `metrics` + serialized snapshot (offline/resume).

3 · Mathematical model

$$\min \sum_{i \in P} \sum_{j \in B} c_{ij} x_{ij} \quad \text{s.t.} \quad \sum_j x_{ij} \leq 1, \quad \sum_i x_{ij} \leq 1, \quad x_{ij} \in \{0, 1\}$$

- $c_{ij} = \infty \rightarrow$ **hard constraint** (gender / accessibility) = forbidden edge.
- else $c_{ij} =$ **soft penalties**: budget overflow, building / room-type mismatch.
- Pad to square $K \times K$ ($K = \max(|P|, |B|)$); solve **Hungarian** $O(K^3)$.
- **Kuhn bipartite matching** recovers feasible placements the optimum stranded.



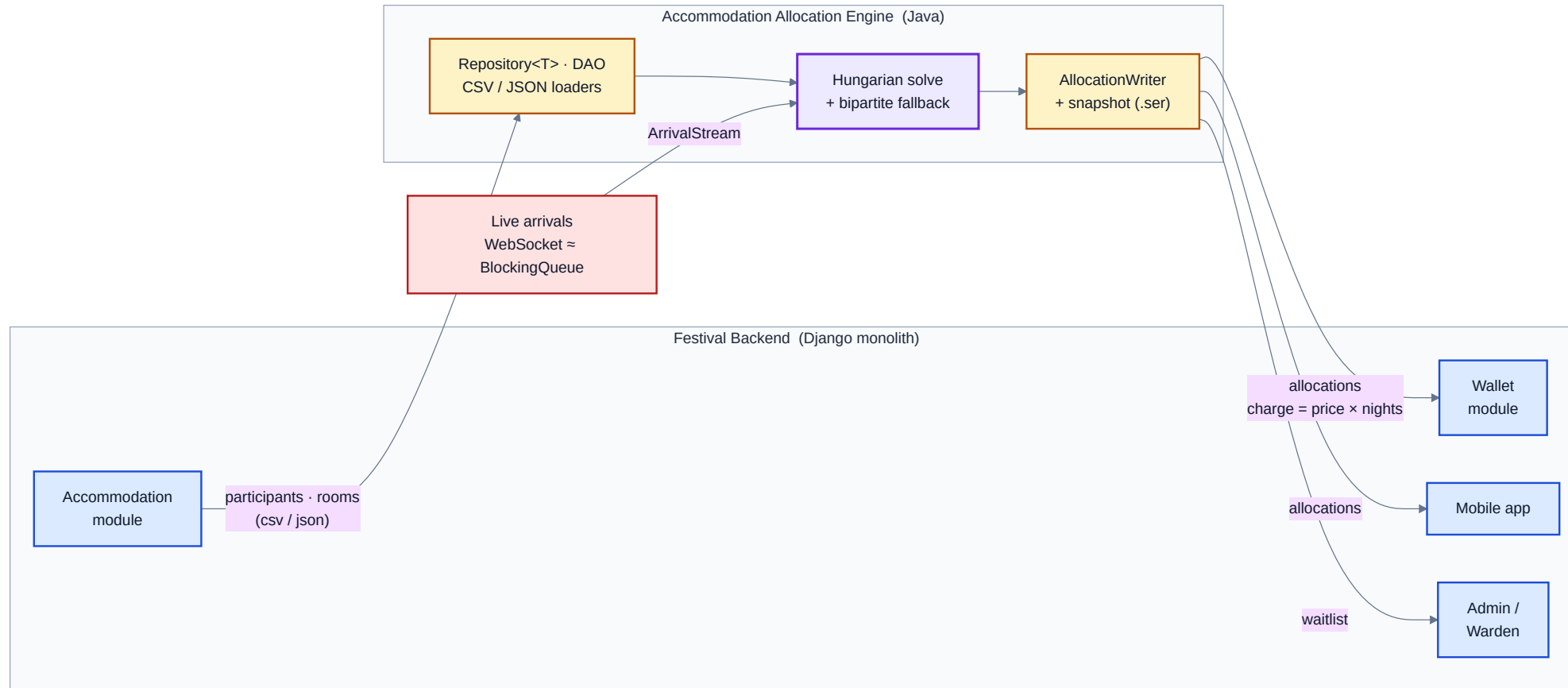
4 · Innovation

- **Capacity via bed-expansion + dummy padding** turns a many-to-one, unequal-size problem into a clean square assignment the Hungarian algorithm solves exactly.
- **Priority as a matrix-only bias:** scarce beds go to higher-priority categories first, *without* distorting room choice or the reported dissatisfaction — a subtle, principled trick.
- **Two-phase optimise-then-repair:** cost-optimal Hungarian, then bipartite matching for feasibility, then a cost-neutral **roommate co-location** local search.
- **Real-time ready:** a `BlockingQueue` arrival stream re-allocates late registrations live.

5 · Success metrics

Metric	Target
Hard-constraint violations	0 (guaranteed)
Placement rate	100 % when capacity $\geq$ demand
Total / avg dissatisfaction	minimised (global optimum)
Bed utilisation	maximised
Runtime @ N $\approx$ 500	< 0.3 s (measured 0.23 s; N=800 $\approx$ 0.6 s)
Waitlist fairness	strictly by category, then arrival day

6 · Integration with the platform

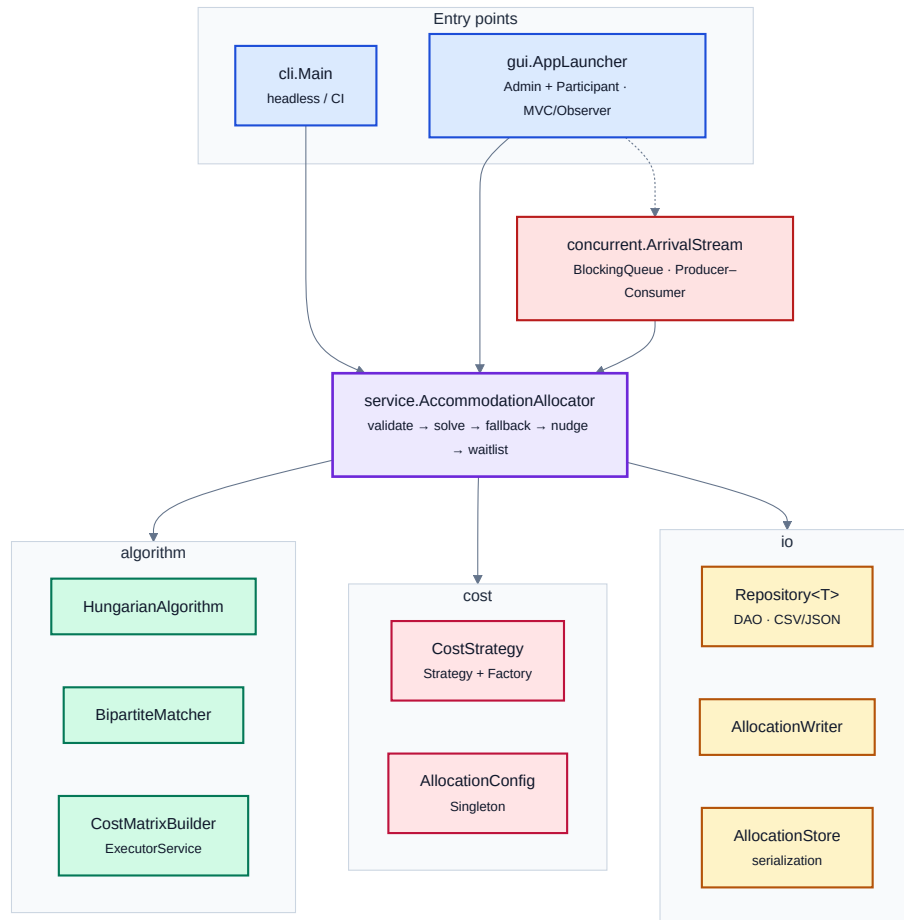


File-exchange contract (CSV and JSON, matching Django REST payloads); a `BlockingQueue` arrival stream mimics the WebSocket feed; `.ser` snapshot for offline cache / resume.

7 · Advanced Java features (and why)

Feature	Use
Generics	<code>Repository<T></code> , <code>CostStrategy</code> — type-safe DAO & cost models
Concurrency	<code>ExecutorService</code> (parallel cost matrix), <code>SwingWorker</code> , <code>BlockingQueue</code> Producer-Consumer
Collections	<code>PriorityQueue</code> waitlist, <code>TreeMap</code> / <code>HashMap</code> / <code>EnumMap</code>
Serialization	<code>ObjectOutputStream</code> snapshot (offline/resume)
File I/O	buffered CSV + Gson JSON, try-with-resources
Exceptions	checked hierarchy for invalid/infeasible/load errors
Design patterns	Strategy, Factory, DAO, Observer, MVC, Singleton, Builder, Producer-Consumer

8 · Architecture



- **Core engine** (algorithm + service) — pure, unit-tested, GUI-independent.
- **DAO layer** (io) — CSV/JSON in, CSV/JSON/ .ser out.
- **CLI** — scriptable, CI-friendly.
- **Swing GUI** — **two roles**: Admin dashboard + Participant view.
- **37 JUnit 5 tests**, README, demo video.

Full pipeline, UML & GUI diagrams: see [diagrams/](#) .

9 · Feasibility & scope (2–3 weeks)

- **Week 1** — domain model, Hungarian core, cost strategy, unit tests.
- **Week 2** — IO (CSV/JSON), allocator orchestration, waitlist, serialization, CLI.
- **Week 3** — Swing GUI (two roles), arrival stream, polish, demo + report.

Self-contained, no server required; runs from sample data out of the box. Realistic, bounded, and directly useful to the festival's Accommodation operations.

Thank you

Accommodation Allocation Engine

Optimal, fair, constraint-safe room assignment — integrated with Wallet, Mobile & Accommodation.

Questions?